ChatGPT

OpenMake LLM 고도화 보고서: Open WebUI 벤치마크 기반 개선안

결론

OpenMake LLM의 고도화는 **"Wide & Shallow(통합 폭)"을 따라가기보다, 현재 강점인 "Narrow & Deep(파이프라인 깊이)"를 유지하면서 '엔터프라이즈 신뢰성(Trust)' 결손을 메우는 방향**이 가장 방어 가능하고 현실적이다. 이 접근은 (1) 보안·권한·감사로 법/규제 리스크를 먼저 낮추고, (2) RAG·문서처리 정밀도를 올려 품질/운영 리스크를 줄이며, (3) 배포·관측성·확장 포인트를 표준화해 비용을 통제한다. Open WebUI가 "프로토콜-퍼스트 + 플러그인/파이프라인 생태계"로 폭을 확장하는 동안, OpenMake LLM은 **정확도·재현성·보안성·운영성**을 전면으로 세워 차별화해야 한다. [1]

아래 로드맵은 "기능 나열"이 아니라 **제품 리스크를 가장 크게 줄이는 단일 우선순위**에 맞춰 구성했다.

우선순위	목표(한 줄)	왜 지금(리스크 관점)	핵심 산출물(구현 단위)	수용 기준(운영 지표)
P0	**SSRF/외부페치 안전화 + 객체 단위 권한(BOLA) 방어**	웹 페치/스크래핑/리서치 기능은 SSRF·내부망 탐색·메타데이터 유출로 직결(법/규제·사고 비용 최상위). Open WebUI도 유사 공격면에서 SSRF 취약점이 실제로 보고됨.	URL 처리 유틸(allowlist+IP block+DNR/DNS rebinding 방어), egress 제어 가이드, 리소스 전 엔드포인트 BOLA 체크 공통 미들웨어	보안 테스트 시 RFC1918/메타데이터(예: 169.254.169.254) 접근 차단, 임의 URL 페치 불가, 권한 없는 객체 접근 0건
P1	**Knowledge Base(컬렉션/공유/권한) + RLS 기반 멀티테넌시**	"파일 단위 RAG"는 팀/조직 확장 시 즉시 한계. 데이터 권한은 운영·책임소재를 결정(감사/분쟁 대응).	컬렉션 모델(다대다), AccessGrants/Group(또는 최소한 Role+Policy), Postgres RLS 정책	1) 사용자/그룹별 KB 공유·권한 재현 가능, 2) 감사 로그로 truyback 가능
P1	**RAG 정밀도: 하이브리드(lexical+dense) + 리랭킹**	Dense-only는 정확한 키워드/고유명사에서 실패율이 높고, 리랭킹이 최종 품질을 좌우한다는 근거가 축적됨.	Postgres FTS(tsvector/GIN) + pgvector 결합, RRF/score fusion, Cross-encoder 리랭커	고정 벤치에서 정답 포함률·MRR 개선, 사용자 불만("찾을 수 없음/엉뚱한 근거") 감소

우선순위	목표(한 줄)	왜 지금(리스크 관점)	핵심 산출물(구현 단위)	수용 기준(운영 지표)
P2	**문서/OCR 파이프라인 다중 엔진화 + 품질 게이트**	PDF/스캔/표/수식은 단일 OCR로 품질 편차가 커 운영 이슈로 전환됨.	assessTextQuality 게이트, Tika/Docling/클라우드 OCR 플러그인 슬롯, 비전 기반 추출 옵션	문서 유형별(텍스트PDF/스캔PDF/슬라이드) 추출 성공률·정확도 기준 통과
P2	**Docker 배포 + 관측성(OpenTelemetry) 표준화**	설치 장벽/운영 가시성 부족은 상용화 병목.	docker-compose 템플릿, OTEL trace/metric, 비용·토큰·도구 호출 계측	재현 가능한 배포, 장애 원인 분석 리드타임 단축

벤치마킹 관찰과 제품 포지셔닝 판단

Open WebUI는 공식 문서에서 **오프라인 동작, Ollama 및 OpenAI 호환(Chat Completions) 프로토콜 지원, 프로토콜-퍼스트를 통한 provider-agnostic**을 명시한다. [2] 또한 "Tools/Functions/Pipelines"로 확장을 제공하며, 특히 Pipelines는 **OpenAI API 스펙을 지원하는 UI 클라이언트 어디서든 동작하는 모듈형 워크플로우**라는 방향을 제시한다. [3]

반면, 제공된 내부 분석(프라이빗 코드베이스 관찰)에서 OpenMake LLM은 프로필 기반 동적 파이프라인, A2A 토론, Deep Research 등 **깊은 오케스트레이션을 핵심 가치로** 설정했다. 이 차이는 전략적으로 "누구를 이기려는가"가 아니라 "무엇을 소유하려는가"에 가깝다. Open WebUI가 폭을 확장하는 동안 OpenMake LLM이 폭 경쟁을 그대로 추종하면, (1) 유지보수 비용이 급증하고, (2) 파이프라인 깊이의 차별점이 희석되며, (3) 보안·권한·감사 같은 엔터프라이즈 요구 충족이 지연될 가능성이 높다. [4]

특히 "웹 페치/스크래핑/리서치"가 포함된 제품은 SSRF가 구조적으로 따라온다. 실제로 Open WebUI는 `/api/v1/retrieval/process/web` 관련 SSRF 취약점이 보안 권고(GHSA) 및 CVE로 공개된 바 있다. [5] 이는 OpenMake LLM에도 **동일한 공격면이 존재할 개연성이 높다**는 의미이며, 기능 확장보다 먼저 "안전한 기본값"이 제품의 방어력을 결정한다. SSRF가 무엇이며 어떤 피해(내부 서비스 접근, 클라우드 메타데이터 탈취)가 가능한지는 OWASP가 정리한다. [6]

RAG 검색 정확도 개선: 하이브리드·리랭킹·평가 루프

제공된 내부 분석에 따르면 OpenMake LLM은 pgvector 기반 dense 검색과 시맨틱 캐시를 강점으로 가져가지만, **sparse(BM25/lexical)·하이브리드·리랭킹 부재**가 RAG 정밀도 병목으로 지목된다. 이 지적은 검색 연구/벤치마크 관점에서도 타당하다. BEIR(광범위 IR 벤치마크)는 BM25가 강한 베이스라인임을 보여주고, **리랭킹·late-interaction 계열이 평균적으로 높은 성능을 내는 대신 비용이 증가**한다는 관찰을 제시한다. [7] BM25 자체는 확률적 관련성 프레임워크 기반의 대표적 기법으로 널리 사용되며, Robertson의 정리 문헌과 TREC에서의 Okapi 계열 결과가 장기간 인용돼 왔다. [8]

따라서 OpenMake LLM의 RAG 고도화는 "벡터 DB 다변화"가 아니라 **PostgreSQL 안에서 하이브리드 + 리랭킹을 완성하는 것**이 가장 비용 효율적이다. 이미 pgvector는 IVFFlat/HNSW 등 근사 최근접 탐색 인덱스를 제공하며, 성능-재현율 트레이드오프를 공식 문서/프로젝트에서 설명한다. [9] 또한 PostgreSQL은 내장 Full Text Search(FTS)로 `tsvector/tsquery` 및 랭킹/인덱싱(GIN)을 제공한다. [10]

권고 아키텍처는 2-스테이지(또는 3-스테이지)다.

첫 단계는 **후보군 생성(candidate generation)**이다. 동일 문서 청크에 대해 (a) dense(pgvector cosine) topK, (b) lexical(FTS) topK를 각각 뽑고, 두 랭킹을 결합한다. 결합 방식은 구현 난이도 대비 효과가 검증된 RRF(Reciprocal Rank Fusion)를 기본값으로 두는 것이 안전하다. RRF가 여러 IR run을 결합했을 때 최상 성능을 자주 상회한다는 보고가 있으며, 하이브리드 퓨전 함수 분석 연구도 RRF/score combination을 비교한다. [11]

둘째 단계는 **리랭킹(reranking)**이다. Cross-encoder(BERT) 기반 passage re-ranking은 "BM25로 대량 후보 → 재정렬" 구조에서 성능을 크게 끌어올릴 수 있음이 고전적으로 보고되었다. [12] 여기서 비용은 후보군 크기(예: 50~200)로 제어한다. 이는 OpenMake LLM이 이미 갖고 있는 "Cost Tier / Complexity Assessor" 철학(내부 분석상의 강점)과도 정합적이다.

셋째(선택) 단계는 **생성 전 근거 품질 관리**다. "retrieve는 됐는데 근거가 약함"을 줄이기 위해, RAG 평가 루프를 제품에 내장해야 한다. RAGAs는 참조 없이도 RAG 파이프라인을 평가하기 위한 프레임워크를 제안한다. [13] 그리고 retrieval 자체는 BEIR 같은 이질적 벤치로 "특정 도메인 과적합"을 조기에 감지할 수 있다. [14]

마지막으로, Knowledge Base와 권한이 들어오면 **DB 레벨 멀티테넌시로 검색 누수 방지**가 필요하다. PostgreSQL은 Row Level Security(RLS)와 `CREATE POLICY`를 공식 기능으로 제공한다. OpenMake LLM이 pgvector를 고정 선택한 구조라면, RLS는 "RAG 권한"을 가장 낮은 계층에서 강제하는 데 유리하다. [15]

문서 처리·OCR 고도화: 포맷 커버리지와 품질 게이트

제공된 내부 분석은 OpenMake LLM의 문서 파이프라인이 pdf-parse + OCR(tesseract.js) 폴백 중심이며, "추출 텍스트 길이 < 100" 같은 단순 기준이 품질 이슈를 놓칠 가능성을 P0 문제로 지목한다. 이 문제는 단순 버그가 아니라, 문서 처리에서 흔한 "부분 실패(partial failure)"를 탐지하지 못하는 설계 결함에 가깝다. OCR 엔진인 Tesseract 자체는 LSTM 기반 OCR을 포함하며 다국어를 지원하나, 레이아웃·표·수식·읽기 순서 등 문서 이해에는 한계가 있을 수 있다. [16]

따라서 핵심은 OCR 엔진 교체가 아니라, **(a) 품질 게이트(quality gate), (b) 다중 엔진 슬롯, (c) 비용/데이터 거버넌스**를 한 세트로 만드는 것이다.

품질 게이트는 "길이" 대신 다음 계열 지표를 최소 포함해야 한다: (1) 인쇄 가능 문자 비율, (2) 유니코드 대체문자(`<span_ref_1" box="0.500,0.573,0.520,0.589" />`) 비율, (3) 토큰 다양도/반복도(노이즈 탐지), (4) 페이지별 추출 분포(특정 페이지가 0에 가까우면 실패로 간주), (5) 표/수식/다단 레이아웃 정황(추출 순서 붕괴). 이 단계의 목적은 **실패를 '조용히' 통과시키지 않고**, 다음 엔진으로 안전하게 넘기는 것이다. (이 항목은 공개 출처의 사실이라기보다 제품 설계 권고이므로, 아래의 "다중 엔진" 근거로 보강한다.)

다중 엔진 슬롯은 현실적으로 "로컬-퍼스트 + 옵션형 고정밀"이 가장 안전하다. 예를 들어 Apache Tika는 수백~천 종 파일 형식에서 텍스트/메타데이터를 단일 인터페이스로 추출하도록 설계되었고, 포맷 커버리지 확장에 유리하다. [17] Docling은 레이아웃/테이블/읽기 순서 등 구조화 변환을 목표로 하는 오픈소스 툴킷으로, 복잡 PDF를 "RAG 친화적 표현"으로 바꾸는 역할에 적합하다. [18] 클라우드 옵션으로는 Microsoft [19]의 Document Intelligence(Read model)가 문서 OCR/구조 추출을 제공하며, 정확도가 필요한 엔터프라이즈에서 선택지가 된다. [20] Mistral AI [21]의 Document AI OCR도 PDF에서 텍스트와 구조적 요소(표/수식 등) 추출을 표방하며, 복합 문서에서 대안이 될 수 있다. [22]

다만 클라우드 OCR은 **문서 원문이 외부로 전송**되므로, 기본값으로 켜면 법/규제·고객 계약(데이터 처리) 리스크가 커진다. 때문에 "Enterprise tier + 명시적 동의 + 데이터 보존/삭제 정책 + 감사 로그"가 선행되어야 한다. (클라우드 OCR이 키/엔드포인트 기반 호출이라는 점은 각 문서에서 확인된다.) [23]

보안·권한·감사: 엔터프라이즈 구멍 메우기

OpenMake LLM은 제공된 내부 분석에서 API Key 해싱·타이밍 공격 방어·Scope·티어별 레이트리밋 등 "API 운영 감각"이 강점으로 제시된다. 하지만 **SSRF 방어 부재(명시적)**, **그룹/리소스 권한의 부재**, **협업 리소스(knowledge) 단위 권한의 부족**은 엔터프라이즈로 갈수록 치명적 공백이 된다. SSRF는 OWASP가 대표 서버 공격으로 분류하며, URL을 서버가 대신 요청하는 기능이 있는 경우 클라우드 메타데이터 및 내부망 노출로 이어질 수 있음을 강조한다. [24]

여기서 목표는 "보안 기능을 더한다"가 아니라, **공격면을 제품 설계 수준에서 줄이는 것**이다.

첫째, 웹 페치/스크래핑/리서치 도구는 "임의 URL"을 입력으로 받는 순간 SSRF 표적이 된다. Open WebUI의 SSRF 권고(GHSA 및 CVE)는 인증만 된 사용자라도 임의 URL을 서버가 요청하게 만들 수 있음을 보여준다. [25] OpenMake LLM은 다음 통제를 기본값으로 강제해야 한다: (1) 스킴 allowlist(http/https만), (2) 호스트를 IP로 해석 후 private/reserved/link-local 차단, (3) 리다이렉트 체인 검사, (4) DNS rebinding 완화(해석 결과 캐시+재검증), (5) 프록시/egress 정책 문서화 및 권장 네트워크 분리. OWASP SSRF Cheat Sheet는 allowlist·네트워크 분리·egress 제어 등 완화 전략을 체계적으로 제시한다. [26]

둘째, API 보안은 "인증"보다 "객체 단위 권한(BOLA)"이 더 자주 사고로 이어진다. OWASP API Security Top 10 2023의 API1이 Broken Object Level Authorization을 최상단 리스크로 둔 이유가 여기에 있다. [27] OpenMake LLM이 Research session, Document store, Memory, Audit 등 리소스 테이블을 갖고 있다면, **모든 `/:id` 엔드포인트에 공통 BOLA 가드**를 두고 "소유자/그룹/권한 정책"을 DB에서 강제해야 한다.

셋째, 협업을 위한 Knowledge Base를 도입할 때 "애플리케이션 레벨 권한"만으로는 누수 방지에 취약하다. PostgreSQL의 RLS는 정책을 DB가 강제하므로, API 코드 버그가 있더라도 누수를 줄이는 방어선이 된다. [28] 즉, P1의 Knowledge Base는 "기능 개발"이 아니라 **감사 가능성(누가 어떤 근거를 열람/공유했는지)**까지 포함하는 보안 설계로 접근해야 한다.

운영·배포·DX: 성능-비용-관리성 표준화

Open WebUI의 채택 확산에는 "Docker로 바로 띄우는 경험"이 큰 비중을 차지한다. 공식 Quick Start는 `docker run` 및 볼륨 마운트 기반의 설치/업데이트 절차를 제공한다. [29] 또한 Docker [30]는 Compose를 프로덕션에 적용하는 가이드(다중 compose 파일, 업데이트 절차 등)를 공식 문서로 제공한다. [31] OpenMake LLM이 현재 "Node 직접 배포" 중심이라면, **Dockerfile + compose(최소 postgres 포함) + 예제 env**를 P2가 아니라 사실상 P1에 준해 추진해야 설치 장벽과 운영 변동성을 줄일 수 있다.

관측성은 운영 리스크를 비용으로 바꾸는 핵심 도구다. OpenTelemetry는 Node.js에서 트레이스/메트릭을 수집하는 공식 가이드를 제공하고, Express용 instrumentation 패키지도 배포된다. [32] OpenMake LLM은 이미 "Cost Tier / Rate Limiting / Tool Tiers" 같은 운영 단위를 갖고 있으므로, **(1) 요청 단위 비용(토큰/외부 API/스크래핑), (2) 파이프라인 단계별 latency, (3) 캐시 히트율(L1/L2), (4) RAG 단계별 recall proxy, (5) 실패 원인 분류(문서 추출, 웹검색, 모델 호출, 도구 호출)**를 OTEL span/metric으로 계측하는 것이 가장 높은 ROI를 제공한다. (계측 자체의 방법론은 OpenTelemetry 문서 범위에서 지원된다.) [33]

마지막으로 생태계 확장은 "프로바이더를 많이 붙인다"가 아니라 **호환성 레이어를 제공한다**가 더 싸고 빠르다. Open WebUI는 OpenAI 호환(Chat Completions) 프로토콜을 전면에 두고, Pipelines 역시 OpenAI API 스펙 기반 워크플로우를 강조한다. [34] OpenAI 호환 서버를 제공하는 오픈소스(예: vLLM)는 "클라이언트 변경 없이 교체 가능한" 서빙 인터페이스가 실제로 생태계 표준처럼 작동함을 보여준다. [35] OpenMake LLM이 이 레이어를 제공하면, 내부적으로는 "깊은 파이프라인"을 유지하면서도 외부 도구·클라이언트·워크플로우와의 결합 비용을 크게 낮출 수 있다.

요약하면, OpenMake LLM 고도화의 핵심은 **기능 확장 경쟁이 아니라, 신뢰성(보안/권한/감사)과 정확도(RAG/문서처리)를 먼저 '제품 기본값'으로 고정하는 것**이다. 그 위에서만 Docker·관측성·호환성 레이어가 폭발적으로 효과를 낸다. [36]

---

[1] [2] [19] [34] Open WebUI: Home
https://docs.openwebui.com/?utm_source=chatgpt.com

[3] [30] Pipelines: UI-Agnostic OpenAI API Plugin Framework
https://docs.openwebui.com/features/extensibility/pipelines/?utm_source=chatgpt.com

[4] [27] OWASP Top 10 API Security Risks – 2023
https://owasp.org/API-Security/editions/2023/en/0x11-t10/?utm_source=chatgpt.com

[5] [25] Server-Side Request Forgery (SSRF) via Arbitrary URL ...
https://github.com/open-webui/open-webui/security/advisories/GHSA-c6xv-rcvw-v685?utm_source=chatgpt.com

[6] [24] Server Side Request Forgery
https://owasp.org/www-community/attacks/Server_Side_Request_Forgery?utm_source=chatgpt.com

[7] [14] BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models
https://arxiv.org/abs/2104.08663?utm_source=chatgpt.com

[8] The Probabilistic Relevance Framework: BM25 and Beyond
https://www.staff.city.ac.uk/~sbrp622/papers/foundations_bm25_review.pdf?utm_source=chatgpt.com

[9] pgvector/pgvector: Open-source vector similarity search for ...
https://github.com/pgvector/pgvector?utm_source=chatgpt.com

[10] Documentation: 18: 12.3. Controlling Text Search
https://www.postgresql.org/docs/current/textsearch-controls.html?utm_source=chatgpt.com

[11] Reciprocal Rank Fusion outperforms Condorcet and ...
https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf?utm_source=chatgpt.com

[12] Passage Re-ranking with BERT
https://arxiv.org/abs/1901.04085?utm_source=chatgpt.com

[13] Automated Evaluation of Retrieval Augmented Generation
https://arxiv.org/abs/2309.15217?utm_source=chatgpt.com

[15] PostgreSQL: Documentation: 18: CREATE POLICY
https://www.postgresql.org/docs/current/sql-createpolicy.html?utm_source=chatgpt.com

[16] Tesseract Open Source OCR Engine (main repository)
https://github.com/tesseract-ocr/tesseract?utm_source=chatgpt.com

[17] Apache Tika – Apache Tika
https://tika.apache.org/?utm_source=chatgpt.com

[18] Docling: An Efficient Open-Source Toolkit for AI-driven ...
https://research.ibm.com/publications/docling-an-efficient-open-source-toolkit-for-ai-driven-document-conversion?utm_source=chatgpt.com

[20] [23] Read model OCR data extraction - Document Intelligence
https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/prebuilt/read?view=doc-intel-4.0.0&utm_source=chatgpt.com

[21] [32] [33] Node.js
https://opentelemetry.io/docs/languages/js/getting-started/nodejs/?utm_source=chatgpt.com

[22] OCR Processor | Mistral Docs
https://docs.mistral.ai/capabilities/document_ai/basic_ocr?utm_source=chatgpt.com

[26] [36] Server-Side Request Forgery Prevention Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html?utm_source=chatgpt.com

[28] Documentation: 18: 5.9. Row Security Policies
https://www.postgresql.org/docs/current/ddl-rowsecurity.html?utm_source=chatgpt.com

[29] Quick Start
https://docs.openwebui.com/getting-started/quick-start/?utm_source=chatgpt.com

[31] Use Compose in production
https://docs.docker.com/compose/how-tos/production/?utm_source=chatgpt.com

[35] OpenAI-Compatible Server - vLLM
https://docs.vllm.ai/en/stable/serving/openai_compatible_server/?utm_source=chatgpt.com